Spring Framework

Spring Data REST

Rappel des objectifs du cours

- Expliquer ce qu’est un REST
- Présenter le fonctionnement de Spring Web & REST
- Rappels sur Web Service REST
- ResponseEntity & Gestion d’erreur
- Validation des données, Gestion d’erreur et internationalisation
- Présentation des contrôleur REST de Spring

Sommaire

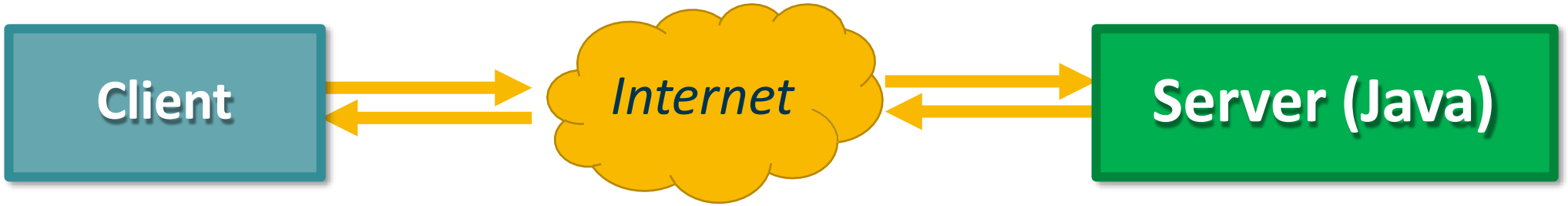
- Introduction à Spring Web et les API REST
- Introduction à Spring Data REST
- @RepositoryRestResource
- Appel à des requêtes spécifiques
- Construire notre API REST avec Spring Web

Spring Framework

Partie 1 – Introduction à Spring Web et les API REST

Introduction à Spring Web

- Un **Web Service** (**WS**) est un programme qui tourne sur un serveur web
- Il renvoie des données à un **client** qui l'interroge
- Le client et le serveur communiquent en utilisant le protocole HTTP

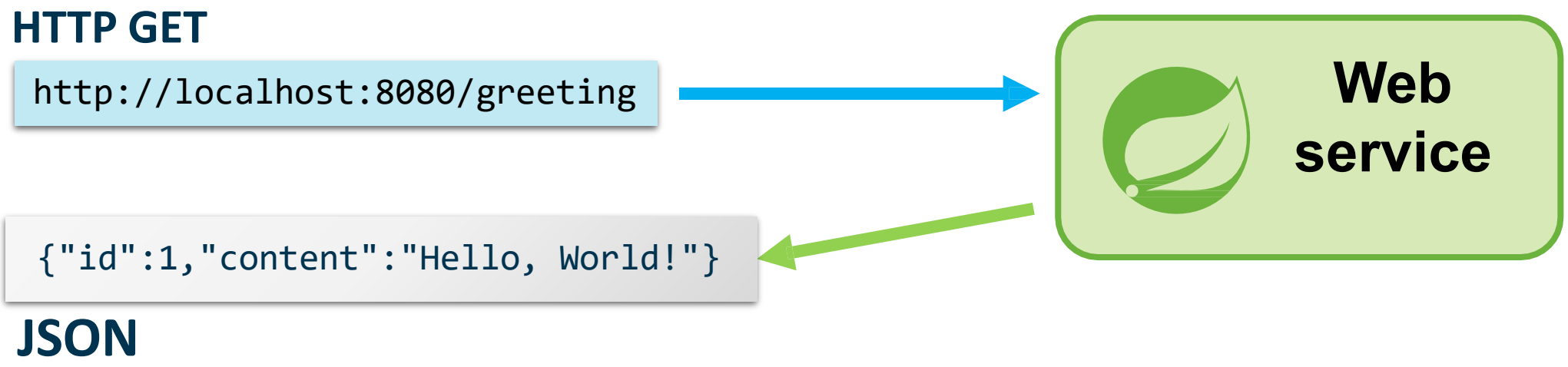


API REST

- REST (**RE**presentational **S**tate **T**ransfer) est une manière de construire notre WS.
- Il utilise les verbes du **protocole HTTP** pour définir des actions précises :
  - **GET** : récupération d'une ressource
  - **POST** : création
  - **PUT** : mise à jour
  - **DELETE** : suppression
- Stateless (le client gère l'état)

Introduction à Spring Web

- Avec Spring Web, nous allons construire un **Web Service RESTful** (qui utilise l'architecture REST)
- Notre programme va pouvoir accepter des requêtes HTTP
- Et il va répondre avec des données en **JSON.**



Introduction à Spring Web

Comparé à Spring MVC :

- On a des 2 verbes HTTP en plus (que GET/POST) :
  - **PUT**
  - **DELETE**
  - **PATCH**
  - **…**
- On **ne renvoie pas une vue** mais des données **JSON** (ou XML, …)
- On n'a pas de concept de vue, et donc pas de concept de modèle

Introduction à Spring Web

- Les différentes url qu'acceptent notre WS sont appelés des **points d'entrée (endpoints)**

- Exemple avec notre projet « Bestioles » :
  - **GET /rest/species** : renvoie la liste des Species
  - **GET /rest/species/2** : renvoie la Species d'id 2
  - **DELETE /rest/species/4** : détruit la Species d'id 4
  - **POST /rest/species** : crée une Species
  - **PUT /rest/species/6** : update la Species d'id 6

Préfixe optionnel
(bonne pratique)

Spring Framework

Partie 2 – Introduction à Spring Data REST

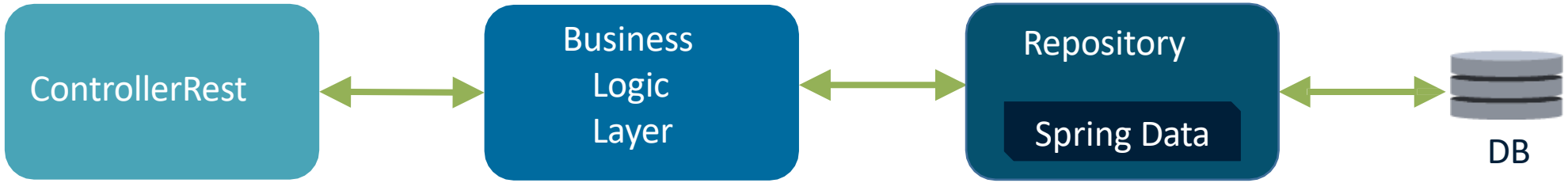
Introduction à Spring Data REST

Spring Data REST fait parti du projet “umbrella Spring Data project”

- Facilite la création de web service REST
- S’appuie sur les principes de REST
- Et d‘HATEOAS (Hypermedia As The Engine of Application State)
  - Une interface universelle adaptable rapidement
  - Le client REST se déplace uniquement dans l’application Web en suivant les URI au format hypermédia
  - Utilise Spring Data, et permet de manipuler toutes ses méthodes

Introduction à Spring Data REST

- Application Web Service REST :



- Application **Spring Data REST** :

Mise en place de Spring Data REST

- Pour utiliser Spring Data REST, il faut ajouter le starter : ***spring-boot-starter-data-rest***

- Dans une application Spring Data REST, il faut créer :
  - Les BO avec des clefs primaires simples
  - Utiliser Spring Data JPA pour mapper vos les entités sur la base de données
  - Créer les Repository de chaque BO
  - Spring Data REST génèrera les contrôleurs automatiquement sur les Repository de Spring Data
  - L'URL sera avec le nom de la classe BO au pluriel (en Anglais)

Exemples BO <-> URL :

- *Article* *<-> http://localhost:8080/articles*
- *Category* *<-> http://localhost:8080/categories*

Spring Data REST – @RepositoryRestResource

Spring Data REST permet de préciser le nom des Ressources :

- Ajout sur les interfaces Repository de l'annotation

  @RepositoryRestResource

- Et des paramètres :

  collectionResourceRel=«nom_ressource»

  path=«nom_ressource_dans_path»

```
@RepositoryRestResource(collectionResourceRel = "gens", path = "les-gens")
public interface PersonRepository extends CrudRepository<Person, Long> {
}
```

Repository

Spring Data REST – Appel de requêtes spécifiques

- **Spring Data REST** permet d'appeler nos recherches spécifiques en ajoutant dans l'URL « /search/nomMethode? »
- Suivi des paramètres de la méthode en tant que paramètres de la requête

```
@RepositoryRestResource(collectionResourceRel = "gens", path = "les-gens")
public interface PersonRepository extends CrudRepository<Person, Long> {
        List<Person> findByLastname(String lastname);
}
```

Repository

```
@RepositoryRestResource(collectionResourceRel = "gens", path = "les-gens")
public interface PersonRepository extends CrudRepository<Person, Long> {
        @RestResource(path = "by-name")
        List<Person> findByLastname(@Param("name") String lastname);
}
```

Attention
les paramètres doivent être de type simple. Pas d'objets !

Spring Framework

Partie 3 – Construire notre API REST avec Spring Web

API REST avec Spring Web – Controleurs

Étape 1 : Écrire nos controllers

- **Intercepte les requêtes HTTP** et **renvoie des réponses en HTTP**
- Typiquement nommés ***<NomEntité>*Controller** (mais pas obligé)

Controller

```
@RestController
@RequestMapping("/rest/person")
public class PersonController {
    { … }
}
```

API REST avec Spring Web – Controleurs

Controller

```
@RestController
@RequestMapping("/rest/person")
public class PersonController {
    { … }
}
```

- **@RestController** : similaire à @Controller mais combine aussi @ResponseBody
- **@ResponseBody** : les données des réponses seront dans le body de la requête HTTP renvoyée.
- **@RequestMapping** : Toutes les URLs commençant par /rest/person seront prises en compte par ce Controller.

Rappel – Les requêtes HTTP

Une requête HTTP est composée de :

- Méthode HTTP (**GET**, **POST**, etc)
- URL
- Paramètres (url.com/?**page=2&size=5**)
- Headers
  - *contient le code de statut de la réponse,*
  - *des méta-données sur la ressource demandée,*
  - *le statut de l'authentification,*
  - *des cookies*
  - *…*
- Body (les données : html, json, binary-data…)

API REST avec Spring Web – Controleurs

Exemple : méthode « find by id »

@GetMapping("/{id}")

@PathVariable

```
@RestController
@RequestMapping("/rest/person")
public class PersonController {
    @GetMapping("/{id}")
    public Person findOne(@PathVariable("id") Integer id) {
        return this.personService.findById(id);
    }
}
```

Controller

- Cette méthode va intercepter **GET rest/person/{id}**

API REST avec Spring Web – Controleurs

Exemple : méthodes create et update

@RequestBody

```
@PostMapping
public Person createPerson(@RequestBody @Valid Person personToCreate) {
    return this.personService.create(personToCreate);
}

@PutMapping
public Person updatePerson(@RequestBody @Valid Person updatedPerson) {
    return this.personService.update(updatedPerson);
}
```

- **@RequestBody** : on demande à avoir un Person dans le body de la requête
- **@Valid** : on valide l'objet envoyé par l'utilisateur.

Controller

API REST avec Spring Web – Couche Service

Dans ces exemples, il y a une **couche service**.

```
return this.personService.update(updatedPerson);
```

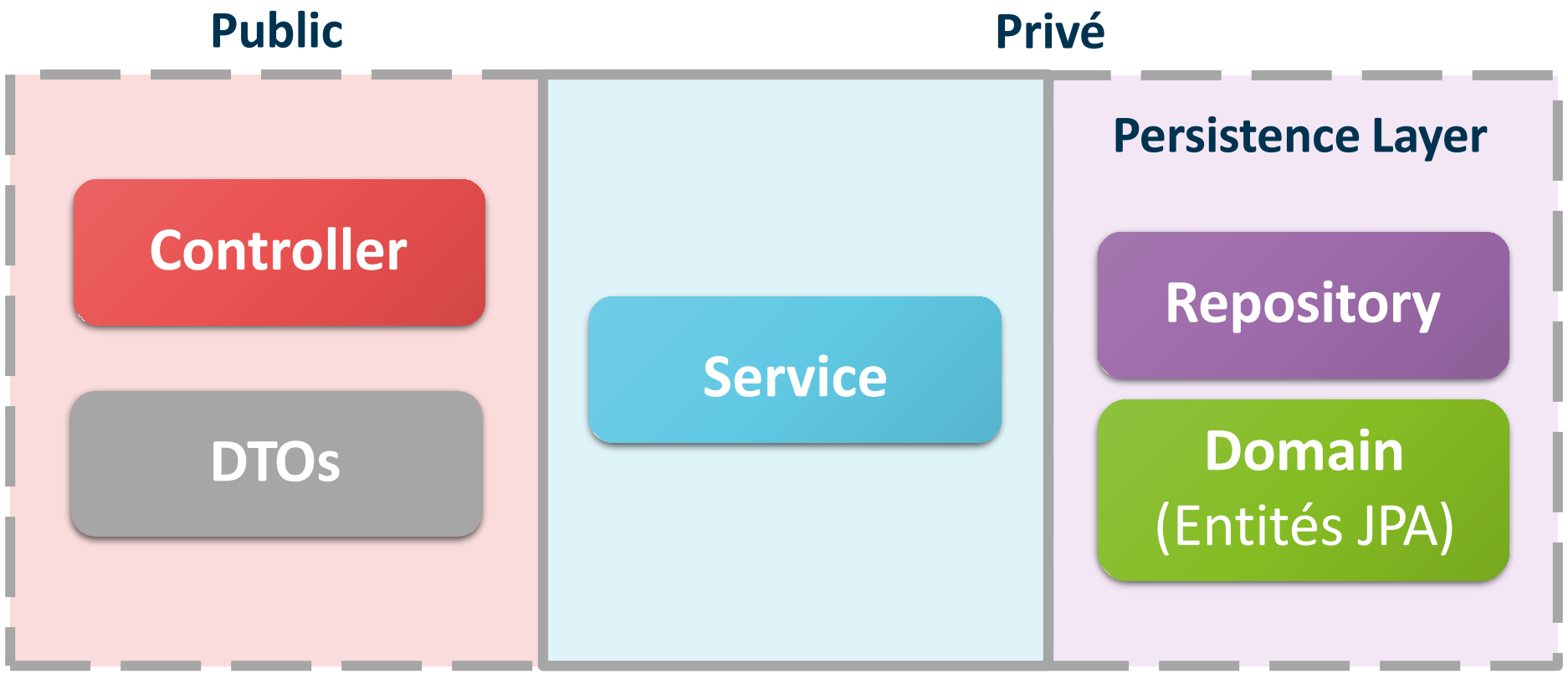
- **C'est une bonne pratique** de séparer les repositories et les controllers par une couche service.

- Rôle 1 = **faciliter la communication entre le controller et le persistence layer.**
- Rôle 2 = **gérer la logique « métier » de notre application**

- Pas forcément requis pour des projets simples !

API REST avec Spring Web – Couche Service

Architecture avec une couche service :



API REST avec Spring Web – Couche Service

Exemple de classe de type service :

@Service

```
@Service
public class PersonService {

    @Autowired
    PersonRepository personRepository;

    public Person create(@Valid Person personToCreate) {  return
        this.personRepository.save(personToCreate);
    }

    public Person update(@Valid Person updatedPerson) {  return
        this.personRepository.save(updatedPerson);
    }

    public Page<Person> findAll(Pageable pageable) {
        return this.personRepository.findAll(pageable);
    }

    public Person findById(Integer id) {
        return  this.personRepository.findById(id).orElseThrow(EntityNotFoundException::new);
    }
    }
```

API REST avec Spring Web – Tester avec Postman

- Pour tester notre controller (et service), il faut donc créer et envoyer des requêtes HTTP à notre programme.

- On va utiliser **Postman** qui est un utilitaire spécifiquement fait pour cela.

- Lien

API REST avec Spring Web – Tester avec Postman

Exemple : Récupérer la Person 6

GET *<url>*

http://localhost:8080/rest/person/6

Save

GET http://localhost:8080/rest/person/6

Params | Authorization | Headers (7) | Body | Pre-request Script | Tests | Settings

Query Params

	KEY	VALUE	DESCRIPTION
	Key	Value	Description

Body | Cookies | Headers (5) | Test Results — 200 OK 20 ms 236 B

Pretty | Raw | Preview | Visualize | JSON

```
1 {
2     "id": 6,
3     "firstname": "Pierre",
4     "lastname": "Sansane",
5     "age": 17,
6     "animals": []
7 }
```

Les données **récupérées** en **JSON**

API REST avec Spring Web – Tester avec Postman

Exemple : créer une Person

On met les données dans le Body

POST *<url>*

http://localhost:8080/rest/person

POST http://localhost:8080/rest/person

Params Authorization Headers (9) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2     "firstname": "Jordi",
3     "lastname": "Mage",
4     "age": 0,
5     "animals": []
6 }
```

Nos données **envoyées** en **JSON**

TP – Webservice RESTful Bestioles

Objectifs :

- Créer un Web service RESTful se basant sur notre projet « Bestioles »
- Le tester avec Postman

Références et Ressources

- Doc Spring – Getting started – RESTful WS